

SATD Detection: Qwen3 vs E5 Embeddings

August 3, 2026

1 What we did

We investigated whether two modern text-embedding models, **Qwen3-Embedding-0.6B** and **E5-Large-v2**, can classify Self-Admitted Technical Debt (SATD) as well as the specialized models used in the original study (GloVe+BiLSTM for identification, fine-tuned BERT for categorization). Texts were embedded by mean-pooling and L2-normalizing the encoder outputs, and a two-step classification scheme was applied per artifact type (code comments, issues, commit messages, pull requests):

1. **Identification** — binary: SATD vs. Not-SATD.
2. **Categorization** — four-class: C/D, REQ, TES, DOC (SATD rows only).

Each task was trained with **XGBoost** and **Logistic Regression** on both embedding sets, giving four combinations per artifact per task. Results were compared against the paper’s baselines (BiLSTM+AugGPT for identification; BERT+AugGPT for categorization), with the supervisor’s replication numbers used for commit messages.

Two independent experiments were run:

1. **Paper-data pipeline** — trained directly on the paper’s pre-augmented dataset ($\approx 109k$ rows; 95,704 after cleaning), which equalizes the four SATD subtypes per artifact while retaining a Not-SATD majority (65–82%). No additional balancing was applied, and the train/validation/test split (80/10/10, stratified) was performed *before* any augmentation to avoid leakage. Logistic Regression used $C = 0.5$.
2. **From-scratch pipeline** — rebuilt from 31 raw source files (820,885 rows; 386,646 after cleaning, deduplication, and label standardization). Minority SATD classes were augmented with a T5 paraphrase model so that all five classes had equal counts per artifact; Not-SATD was downsampled to the same size. Splitting was performed *after* augmentation.

2 Results

Tables 1 and 2 report macro-F1 on the test sets for the best combination (embeddings + XGBoost) of each experiment, alongside the baselines.

Key findings:

- **Categorization beats the BERT baseline.** On code comments, issues, and pull requests, embeddings+XGBoost exceeded the paper’s BERT+AugGPT numbers in *both* experiments (+0.018 to +0.079). This was the only result that was robust across both datasets.

Table 1: Identification (SATD vs. Not-SATD), macro F1.

Artifact	Baseline	Paper-data	Paper-data	From-scratch	From-scratch
		Qwen3	E5	Qwen3	E5
Code comments	0.939	0.9664	0.9587	0.9152	0.8911
Issues	0.878	0.8656	0.8632	0.7981	0.7779
Pull requests	0.862	0.8642	0.8703	0.7924	0.7170
Commit messages	0.910	0.8917	0.8817	0.6861	0.7508

Table 2: Categorization (C/D, REQ, TES, DOC), macro F1.

Artifact	Baseline	Paper-data	Paper-data	From-scratch	From-scratch
		Qwen3	E5	Qwen3	E5
Code comments	0.882	0.9470	0.9581	0.9000	0.9041
Issues	0.899	0.9442	0.9503	0.9410	0.9385
Pull requests	0.876	0.9548	0.9335	0.9270	0.9360
Commit messages	0.980	0.9639	0.9487	0.9589	0.9378

- **Identification is data-dependent.** On the paper’s exact data, identification was baseline-competitive (0.86–0.97; beating the baseline on code comments and pull requests). On the from-scratch dataset it fell below baseline everywhere, dramatically so for commit messages (−0.16 to −0.33, down to 0.58 macro F1).
- **XGBoost consistently beat Logistic Regression** (e.g., paper-data pipeline: 0.9247 vs. 0.87 average macro F1), regardless of embedding model.
- **Qwen3 vs. E5 was effectively a tie.** Averaged across all artifacts and tasks in the paper-data pipeline, E5 scored 0.8988 vs. Qwen3’s 0.8953; per artifact the winner alternated. E5 led slightly on categorization, Qwen3 on identification with XGBoost.
- **Commit messages were the hardest artifact** in both experiments—the only artifact where categorization missed its baseline.
- **Anomaly worth noting:** in the from-scratch pipeline, commit identification (0.58–0.69) was far worse than 4-class categorization on the *same* data (0.94–0.96). A model that distinguishes debt subtypes well yet cannot separate SATD from Not-SATD suggests synthetic-augmentation quality issues, not a genuine model limitation.

3 Takeaways

1. **Modern embeddings + XGBoost are a strong, cheap alternative for SATD categorization.** Beating a fine-tuned BERT baseline on three of four artifacts, with no task-specific fine-tuning, is a reproducible result that held across two completely different data pipelines.
2. **The paper’s identification results depend on its augmentation, not its architecture.** On the paper’s exact augmented data, simple embeddings reproduced the BiLSTM baseline; on from-scratch data augmented with T5, identification collapsed. Since the raw row counts for commit/issue/PR match between the two pipelines, the augmentation generator is

the most plausible source of the gap—suggestive evidence, though confounded by the larger and different from-scratch dataset.

3. **Equalizing class counts does not reproduce the paper’s results.** Forcing all five classes to equal size is a deviation from the paper’s setup (which keeps a Not-SATD majority) and coincided with the worst identification scores—again pointing to synthetic text quality rather than class balance or embedding choice.
4. **Choose XGBoost over logistic regression** for embedding-based classification of these artifacts.
5. **Qwen3 and E5 can be used interchangeably** for this task; if forced to pick, E5-Large had a marginal average edge.
6. **Small artifacts deserve scrutiny.** Commit messages (a few hundred rows per class) are where every method struggles and where augmentation artifacts are most damaging.